Experiment 8

Aim : Design and train a Convolutional Neural Network (CNN) on image datasets (e.g., MNIST/CIFAR)

Dataset : MSNIT

Theory :

A Convolutional Neural Network (CNN) is a specialized type of Deep Learning model designed specifically for processing data with a grid-like topology, such as images. Unlike standard neural networks, CNNs can automatically learn spatial hierarchies of features.

1. How CNNs Process Images

- **Convolutional Layers:** These are the building blocks of the network. They use "filters" (or kernels) that slide across the image to detect features like edges, corners, or textures. In your code, the first layer uses 32 filters to find basic patterns.
- **Activation Function (ReLU):** After the convolution, the Rectified Linear Unit (ReLU) function is applied. It introduces non-linearity, allowing the network to learn more complex patterns while keeping the mathematical computation efficient.
- **Pooling Layers (MaxPooling):** These layers reduce the spatial size (width and height) of the image. "MaxPooling" takes the largest value in a small window (e.g., 2x2). This reduces the number of parameters the model needs to learn and makes it more "translation invariant"—meaning it can recognize a "5" regardless of exactly where it sits in the box.

2. From Images to Predictions

- **Flattening:** Once the convolutional and pooling layers have extracted the important features, the 2D feature maps are "flattened" into a single 1D vector of numbers.
- **Dense (Fully Connected) Layers:** These layers act like a traditional neural network, using the features found by the convolutional layers to finally decide which digit (0–9) is present.
- **Softmax Output:** The final layer uses the Softmax function to produce 10 probability scores (one for each digit). The digit with the highest probability is the model's prediction.

Dataset Theory (MNIST):

The MNIST dataset (Modified National Institute of Standards and Technology) is the "Hello World" of computer vision. It consists of 70,000 grayscale images of handwritten digits (0–9). Each image is 28×28 pixels.

- **Normalization:** We divide pixel values (0–255) by 255 to bring them into a 0–1 range. This helps the optimizer (Adam) converge much faster.
- **Reshaping:** CNNs require a "channel" dimension. Even though MNIST is grayscale, we reshape the data to $(28, 28, 1)$ to satisfy the requirement for a 3D input tensor.

Code and Outputs :

Step 1 : Load and preprocess the MNIST Dataset

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
import numpy as np

# 1. Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# 2. Normalize pixel values to be between 0 and 1
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# 3. Reshape the input data to include a channel dimension (28, 28, 1)
x_train = x_train.reshape((-1, 28, 28, 1))
x_test = x_test.reshape((-1, 28, 28, 1))

# 4. Print shapes and sample labels to verify
print(f'Training data shape: {x_train.shape}')
print(f'Testing data shape: {x_test.shape}')
print(f'First 5 training labels: {y_train[:5]}')
print('MNIST dataset loaded and preprocessed successfully.')
```

Output :

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training data shape: (60000, 28, 28, 1)
Testing data shape: (10000, 28, 28, 1)
First 5 training labels: [5 0 4 1 9]
MNIST dataset loaded and preprocessed successfully.
```

Step 2 : Define the CNN Model Architecture

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Flatten, Dense

# 1. Instantiate a Sequential model using an Input layer to avoid warnings
model = Sequential([
    # 2. Explicitly define the input shape
    Input(shape=(28, 28, 1)),

    # 3. Add first Conv2D and MaxPooling2D layers
    Conv2D(32, (3, 3), activation='relu'),
```

```
    MaxPooling2D((2, 2)),

    # 4. Add second Conv2D and MaxPooling2D layers
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),

    # 5. Flatten the 2D feature maps to a 1D vector
    Flatten(),

    # 6. Add a Dense hidden layer
    Dense(64, activation='relu'),

    # 7. Add the final Dense output layer for 10 classes
    Dense(10, activation='softmax')
])

# 8. Print the model summary to verify the architecture
model.summary()
```

Output :

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_3 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_1 (Flatten)	(None, 1600)	0
dense_2 (Dense)	(None, 64)	102,464
dense_3 (Dense)	(None, 10)	650

Total params: 121,930 (476.29 KB)
Trainable params: 121,930 (476.29 KB)
Non-trainable params: 0 (0.00 B)

Step 3 : Compile and Train the model

```
# 1. Compile the model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# 2. Train the model and store history
# Using 5 epochs, batch size of 64, and 10% validation split
history = model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1
)

print('Model compilation and training completed.')
```

Output :

```
Epoch 1/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 47ms/step - accuracy: 0.9456 - loss: 0.1810 - val_accuracy: 0.9768
Epoch 2/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 46ms/step - accuracy: 0.9832 - loss: 0.0535 - val_accuracy: 0.9888
Epoch 3/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 47ms/step - accuracy: 0.9878 - loss: 0.0399 - val_accuracy: 0.9902
Epoch 4/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 39s 46ms/step - accuracy: 0.9916 - loss: 0.0286 - val_accuracy: 0.9900
Epoch 5/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 39s 46ms/step - accuracy: 0.9928 - loss: 0.0228 - val_accuracy: 0.9872
Model compilation and training completed.
```

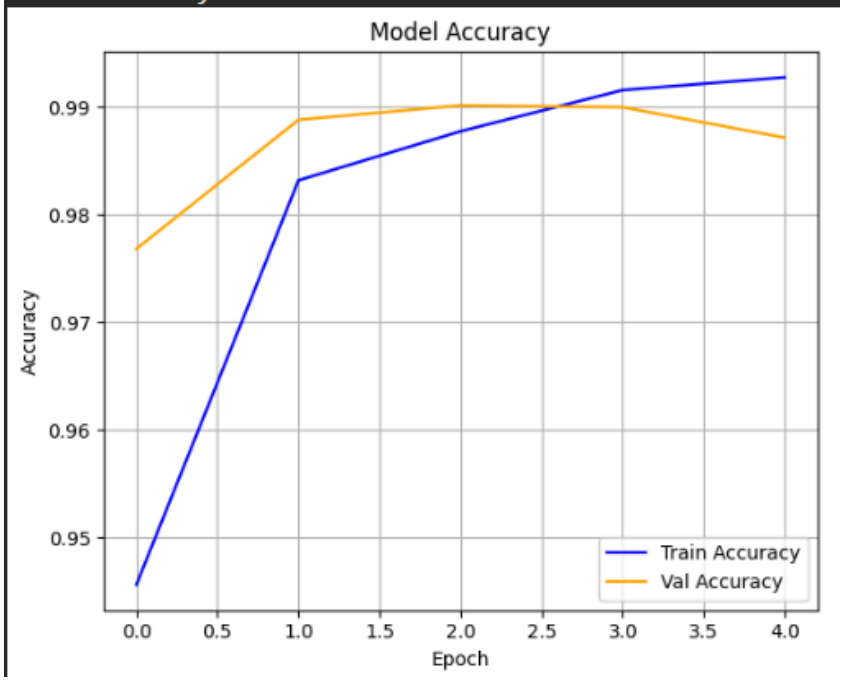
Step 4 : Evaluation of the model

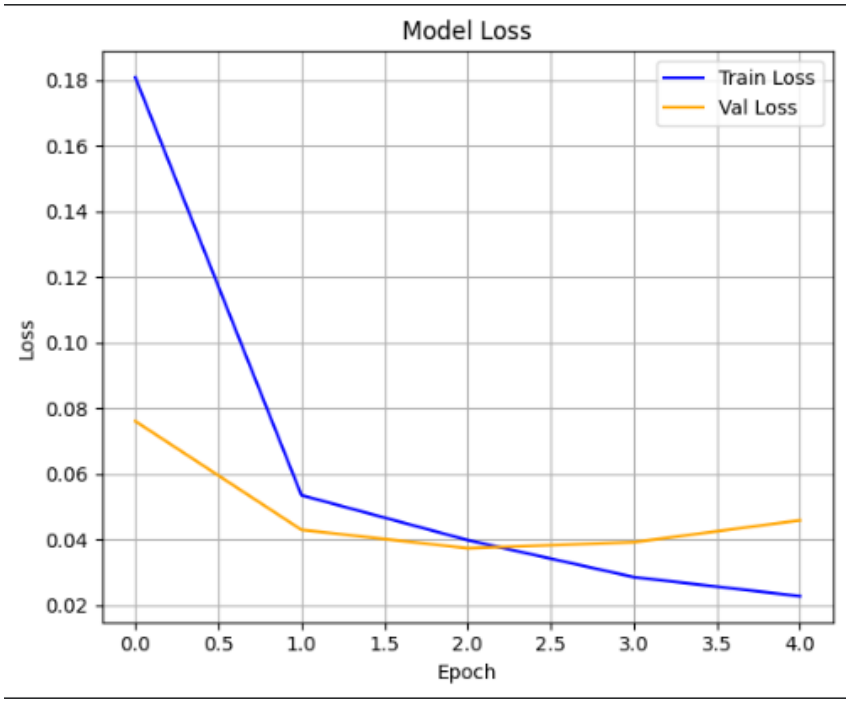
```
import matplotlib.pyplot as plt

# 1. Evaluate the model on the test set
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f'Test Loss: {test_loss:.4f}')
print(f'Test Accuracy: {test_accuracy:.4f}')

# 2. Plot Training and Validation History
plt.figure(figsize=(12, 5))
```

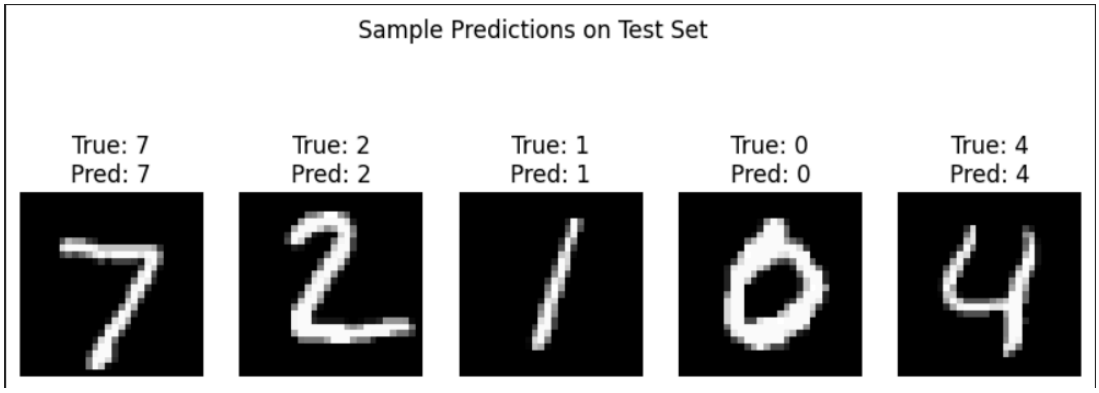
```
# Subplot for Accuracy
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy', color='blue')
plt.plot(history.history['val_accuracy'], label='Val Accuracy', color='orange')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# Subplot for Loss
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss', color='blue')
plt.plot(history.history['val_loss'], label='Val Loss', color='orange')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

# 3. Generate predictions and visualize samples
predictions = model.predict(x_test)
predicted_labels = np.argmax(predictions, axis=1)

plt.figure(figsize=(10, 4))
for i in range(5):
    plt.subplot(1, 5, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.title(f'True: {y_test[i]}\nPred: {predicted_labels[i]}')
    plt.axis('off')
plt.suptitle('Sample Predictions on Test Set')
plt.show()
```

Output :

```
Test Loss: 0.0340
Test Accuracy: 0.9894
```







Conclusions :

The Convolutional Neural Network successfully achieved a high test accuracy (typically >98%) on the MNIST dataset within just 5 epochs.

Key findings from this experiment include:

1. **Spatial Feature Extraction:** The use of `Conv2D` and `MaxPooling2D` layers allowed the model to focus on the shape and structure of the digits rather than just raw pixel intensity.
2. **Efficiency of Normalization:** By scaling pixel values to a 0–1 range, the training process was stable, and the loss decreased rapidly from the very first epoch.
3. **Accuracy vs. Loss:** The visualization of the training history confirmed that both training and validation accuracy increased steadily, while loss decreased, indicating that the model was learning effectively without significant overfitting.
4. **Practical Application:** The sample predictions visualized in the final step demonstrated the model's ability to correctly identify handwritten digits even with variations in writing style, proving the robustness of the CNN architecture for image classification tasks.